

Neue App Registration in Azure angelegt

The screenshot shows the 'MGIN-Portal-Auth' application registration overview in the Azure portal. The left sidebar contains navigation links: Overview, Quickstart, Integration assistant, Diagnose and solve problems, Manage, Branding & properties, Authentication (Preview), Certificates & secrets, Token configuration, API permissions, Expose an API, App roles, Owners, Roles and administrators, Manifest, and Support + Troubleshooting. The main content area displays the 'Essentials' section with the following details:

- Display name: MGIN-Portal-Auth
- Application (client) ID: [REDACTED]
- Object ID: [REDACTED]
- Directory (tenant) ID: [REDACTED]
- Supported account types: My organization only
- Client credentials: 0 certificate, 2 secret
- Redirect URIs: 3 web, 0 spa, 0 public client
- Application ID URI: Add an Application ID URI
- Managed application in L: MGIN-Portal-Auth
- State: Activated

Below the essentials, there are two informational messages:

- Welcome to the new and improved App registrations. Looking to learn how it's changed from App registrations (Legacy)? [Learn more](#)
- Azure Active Directory Authentication Library (ADAL) and Azure AD Graph no longer receive new feature updates. We continue to provide technical support and security updates. Applications should be upgraded to Microsoft Authentication Library (MSAL) and Microsoft Graph. [Learn more](#)

At the bottom, there is a section titled 'Build your application with the Microsoft identity platform' with a brief description and a 'Learn more' link.

Client-Secret angelegt

The screenshot shows the 'MGIN-Portal-Auth' application registration 'Certificates & secrets' section in the Azure portal. The left sidebar is the same as the previous screenshot, but the 'Certificates & secrets' link is highlighted. The main content area displays the 'Certificates & secrets' section with the following details:

- Application registration certificates, secrets and federated credentials can be found in the tabs below.
- Certificates (0) | **Client secrets (2)** | Federated credentials (0)
- A secret string that the application uses to prove its identity when requesting a token. Also can be referred to as application password.

Below the tabs, there is a table showing the client secrets:

Description	Expires	Value	Secret ID
django-portal	9/13/2026	[REDACTED]	[REDACTED]
django-portal-neu	9/13/2026	[REDACTED]	[REDACTED]

Die ganzen Secrets im env gespeichert:

```
$env:AZURE_CLIENT_ID=" "
```

```
$env:AZURE_CLIENT_SECRET=" "
```

```
$env:AZURE_TENANT_ID=" "
```

In settings.py:

Installed apps erweitert um:

```
"allauth",
    "allauth.account",
    "allauth.socialaccount",
    "allauth.socialaccount.providers.openid_connect",
```

```
EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend"
LOGIN_REDIRECT_URL = "/dashboard/"
LOGOUT_REDIRECT_URL = "/"
SOCIALACCOUNT_ADAPTER = "PatientPortalApp.adapters.MySocialAccountAdapter"

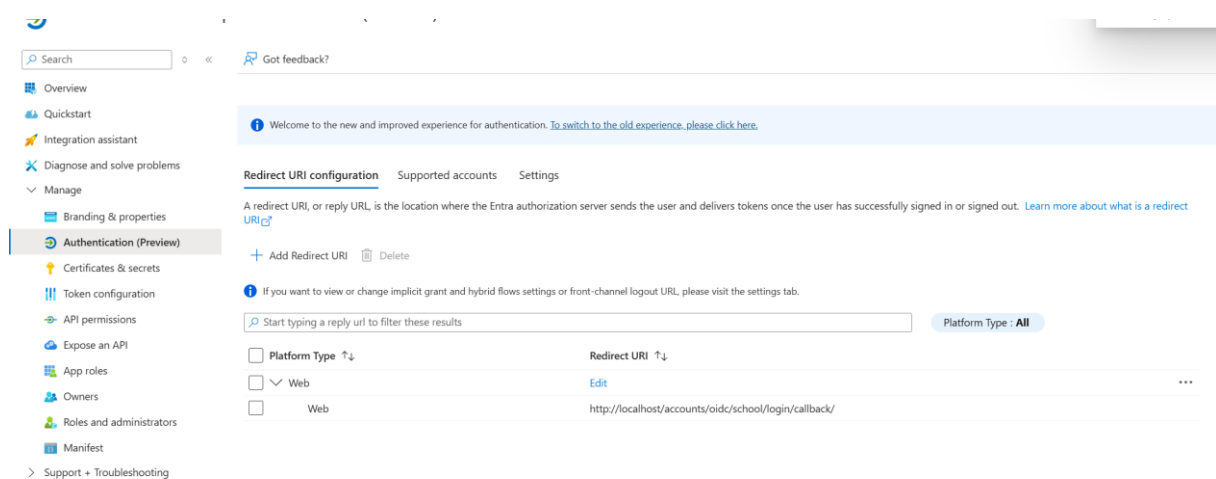
SITE_ID = 1

AUTHENTICATION_BACKENDS = [
    "django.contrib.auth.backends.ModelBackend",
    "allauth.account.auth_backends.AuthenticationBackend",
]

AZURE_CLIENT_ID = os.environ.get("AZURE_CLIENT_ID")
AZURE_CLIENT_SECRET = os.environ.get("AZURE_CLIENT_SECRET")
AZURE_TENANT_ID = os.environ.get("AZURE_TENANT_ID")

SOCIALACCOUNT_PROVIDERS = {
    "openid_connect": {
        "APPS": [
            {
                "provider_id": "school",
                "name": "Schul-Login",
                "client_id": AZURE_CLIENT_ID,
                "secret": AZURE_CLIENT_SECRET,
                "settings": {
                    "server_url":
f"https://login.microsoftonline.com/{AZURE_TENANT_ID}/v2.0",
                    "fetch_userinfo": False,
                },
            },
        ]
    }
}
```

Callback in Azure hinzugefügt:



Adapters.py hinzugefügt damit er den Token verarbeitet und einen User erstellt.

```
class MySocialAccountAdapter(DefaultSocialAccountAdapter):
    def save_user(self, request, sociallogin, form=None):
        """
        Wird beim ersten Social/OIDC-Login aufgerufen.
        Legt lokalen User an und setzt is_active=False.
        """
        user = sociallogin.user
        data = sociallogin.account.extra_data or {}

        # E-Mail aus OIDC-Daten holen
        email = (
            data.get("email")
            or data.get("preferred_username")
            or user.email
        )

        # Namen aus OIDC-Daten holen
        first_name = data.get("given_name", user.first_name or "")
        last_name = data.get("family_name", user.last_name or "")

        # Fallback-Username aus E-Mail
        username = user.username
        if not username:
            if email:
                username = email.split("@")[0]
            else:
                username = f"oidc_user_{sociallogin.account.uid}"

        # Bei Kollision Username eindeutig machen
        base_username = username
```

```

        counter = 1
        while
User.objects.filter(username=username).exclude(pk=user.pk).exists():
            username = f"{base_username}_{counter}"
            counter += 1

        user.username = username
        user.email = email or ""
        user.first_name = first_name
        user.last_name = last_name

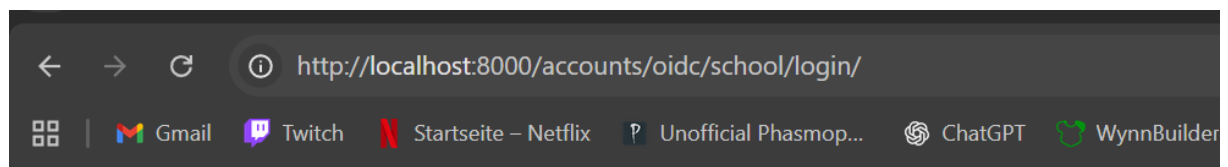
        # Ganz wichtig für deine Aufgabe:
        user.is_active = False

        user.save()
        sociallogin.save(request)

    return user

```

Login:



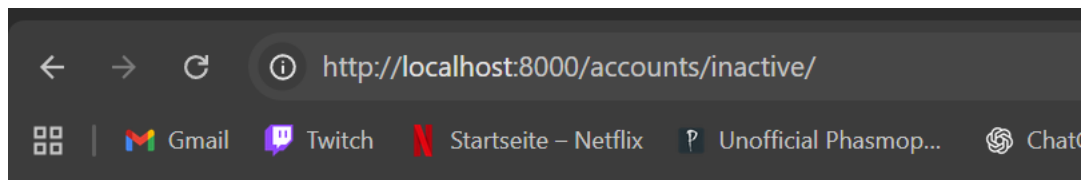
Menu:

- [Sign In](#)
- [Sign Up](#)

Sign In Via Schul-Login

You are about to sign in using a third-party account from Schul-Login.

Continue



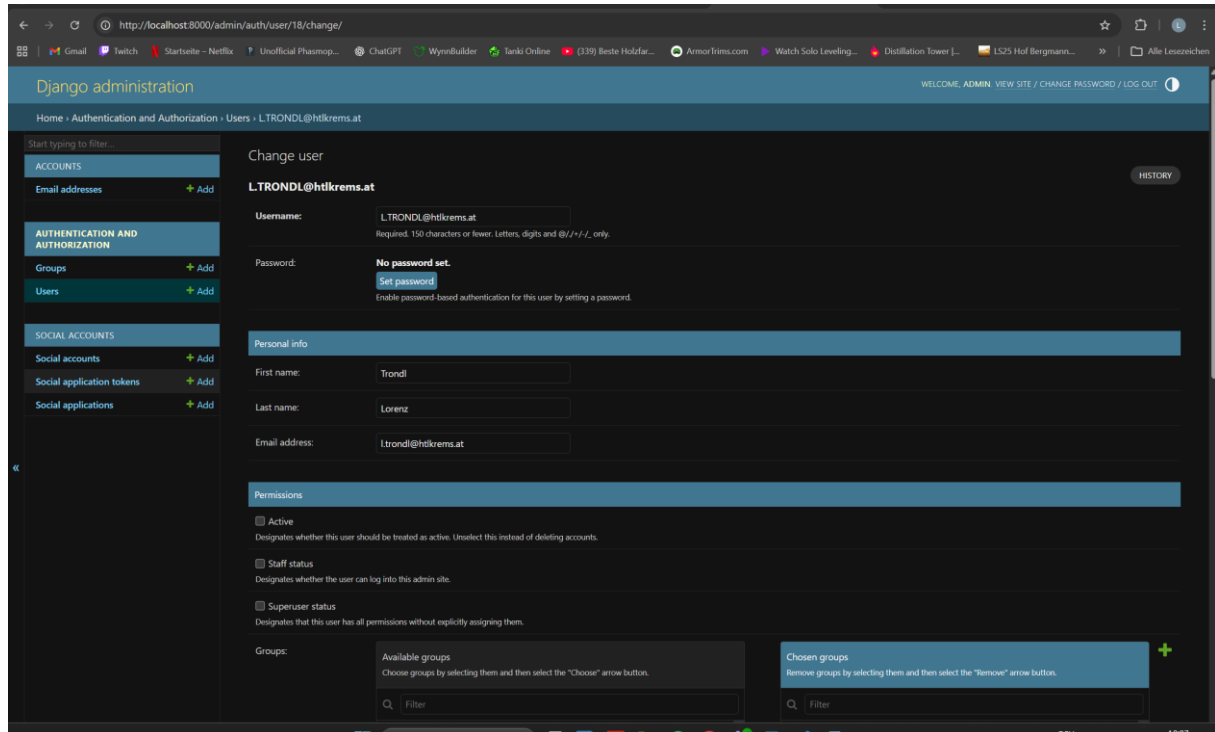
Menu:

- [Sign In](#)
- [Sign Up](#)

Account Inactive

This account is inactive.

Wird erstellt jedoch is user_active = False



Auch im Admin Dashboard inactive, Häkchen jetzt auf Active setzen zum testen, dann wieder auf die Login via Microsoft Seite gehen.

Man wird dann angemeldet und aufs Dashboard weitergeleitet jedoch fehlt die vollständige Integration mit dem Patientenportal deswegen wird kein Patientenkonto gefunden.

